



Bài 5

ĐIỀU PHỐI ĐA DỊCH VỤ VỚI

DOCKER COMPOSE & READINESS CHECK

Giảng viên: ThS. Nguyễn Văn Nhân

Email: nhannv@dainam.edu.vn

Điện thoại: 0346542854

1. Giải thích được service discovery trong Docker Compose, tức là các service gọi nhau bằng tên service thay vì IP
2. Phân biệt được depends_on thường và depends_on có condition: service_healthy, để hiểu vì sao “container đã chạy” chưa chắc “service đã sẵn sàng”.
3. Phân biệt hai tầng network: team-internal vs class-net
4. Hiểu các cách đóng gói AI service



1. Đặt vấn đề
2. Docker Compose
3. Service discovery & thứ tự khởi động
4. Thực hành





ĐẠI NAM
UNIVERSITY

ĐẶT VẤN ĐỀ



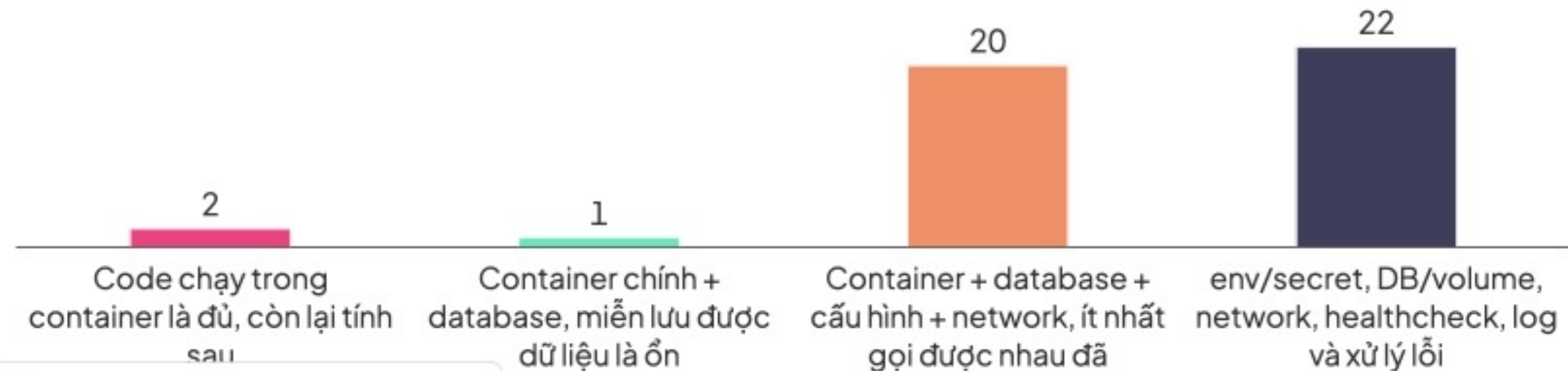


Join at:
[ahaslides.com/
LVU3F](https://ahaslides.com/LVU3F)

To join, go to: ahaslides.com/LVU3F

AhaSlides

Đến giờ demo, service của nhóm bạn docker run lên rất tự tin. Nhưng để nhóm khác gọi được mà không “toang”, theo nhóm bạn service cần gì?



✓ Presentation selected for PowerPoint

≡ < 1 > 🗨

👍👎👏 45 👤 8 / 50 ✓

^ Get Feedback



Đến giờ demo, code thì đã viết xong, Docker cũng đã chạy, nhưng đến đoạn gọi API analysis thì cả nhóm khựngg lại:

‘Ờ... URL của nhóm analysis là gì nhỉ?’



TIẾN ĐỘ DỰ ÁN

- ✓ Code hoàn thành
- ✓ Build thành công
- ✓ Docker chạy ổn
- Gọi API analysis (???)

AnalysisService.java

```
1 public class AnalysisService {
2     public AnalysisResponse run(AnalysisRequest req) {
3         HttpHeaders headers = new HttpHeaders();
4         headers.setContentType(MediaType.APPLICATION_JSON);
5         HttpEntity<AnalysisRequest> entity = new HttpEntity<>(req, headers);
6         ResponseEntity<AnalysisResponse> response =
7             restTemplate.postForEntity(analysisUrl, entity, AnalysisResponse.class);
8         return response.getBody();
9     }
10 }
```

✓ CODE HOÀN THÀNH

TERMINAL

```
$ docker ps
CONTAINER ID   IMAGE          STATUS    PORTS
a1bc234de56f9  analysis-service Up 2 minutes 0.0.0.0:8080->8080/tcp
b2c3d4e5f6a7   mysql         Up 2 minutes 0.0.0.0:3306->3306/tcp
c3d4e5f6a7b8   redis        Up 2 minutes 0.0.0.0:6379->6379/tcp
✓ All containers are running
```



API REQUEST

⚠ THIẾU URL

URL

POST

https://.../analysis/run



Headers

Body

Auth

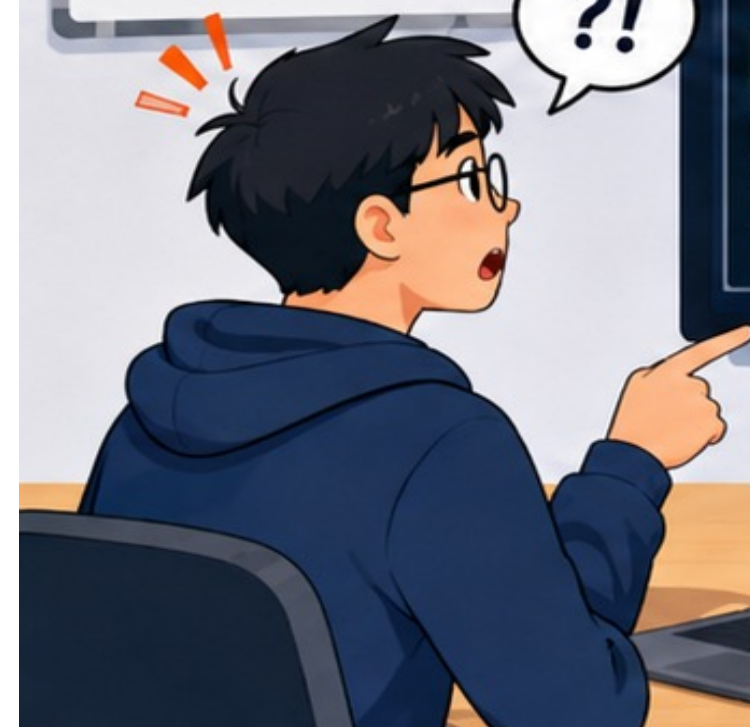
```
{
  "orderId": "ORD-12345",
  "amount": 500000,
  "currency": "VND"
}
```

URL ở đâu
nhỉ ???

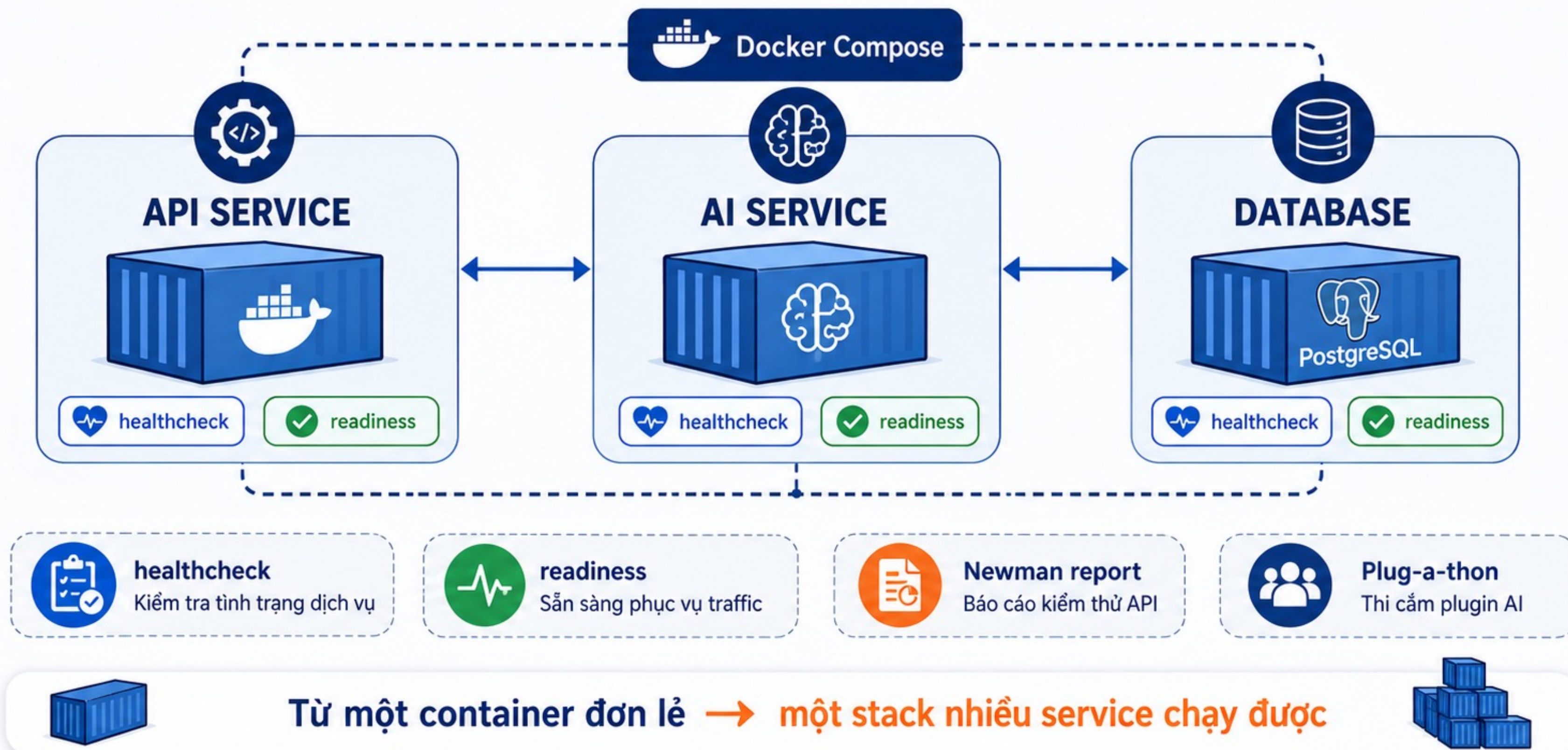
?!

??

?

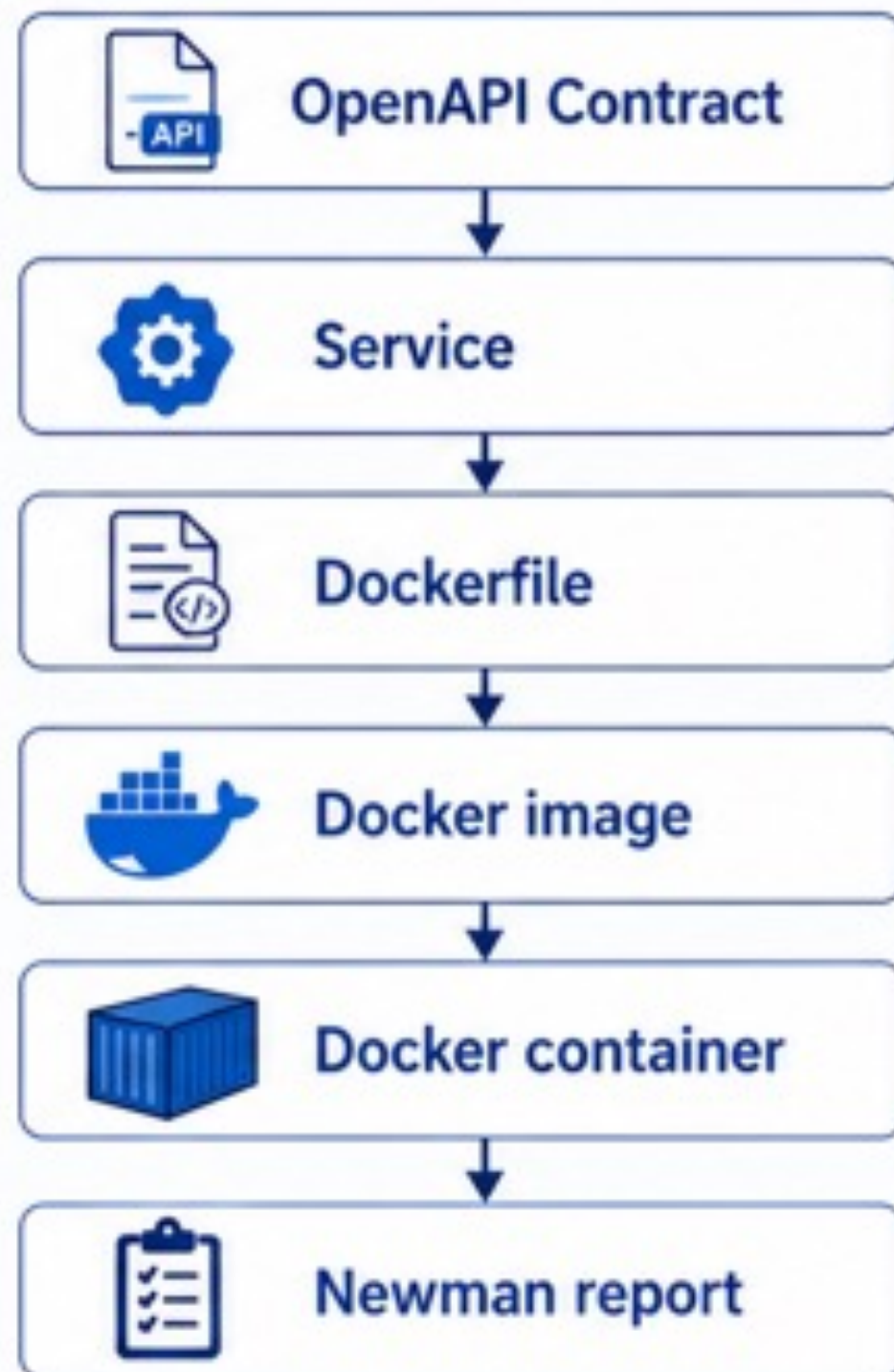


ĐIỀU PHỐI ĐA DỊCH VỤ TRONG SMART CAMPUS



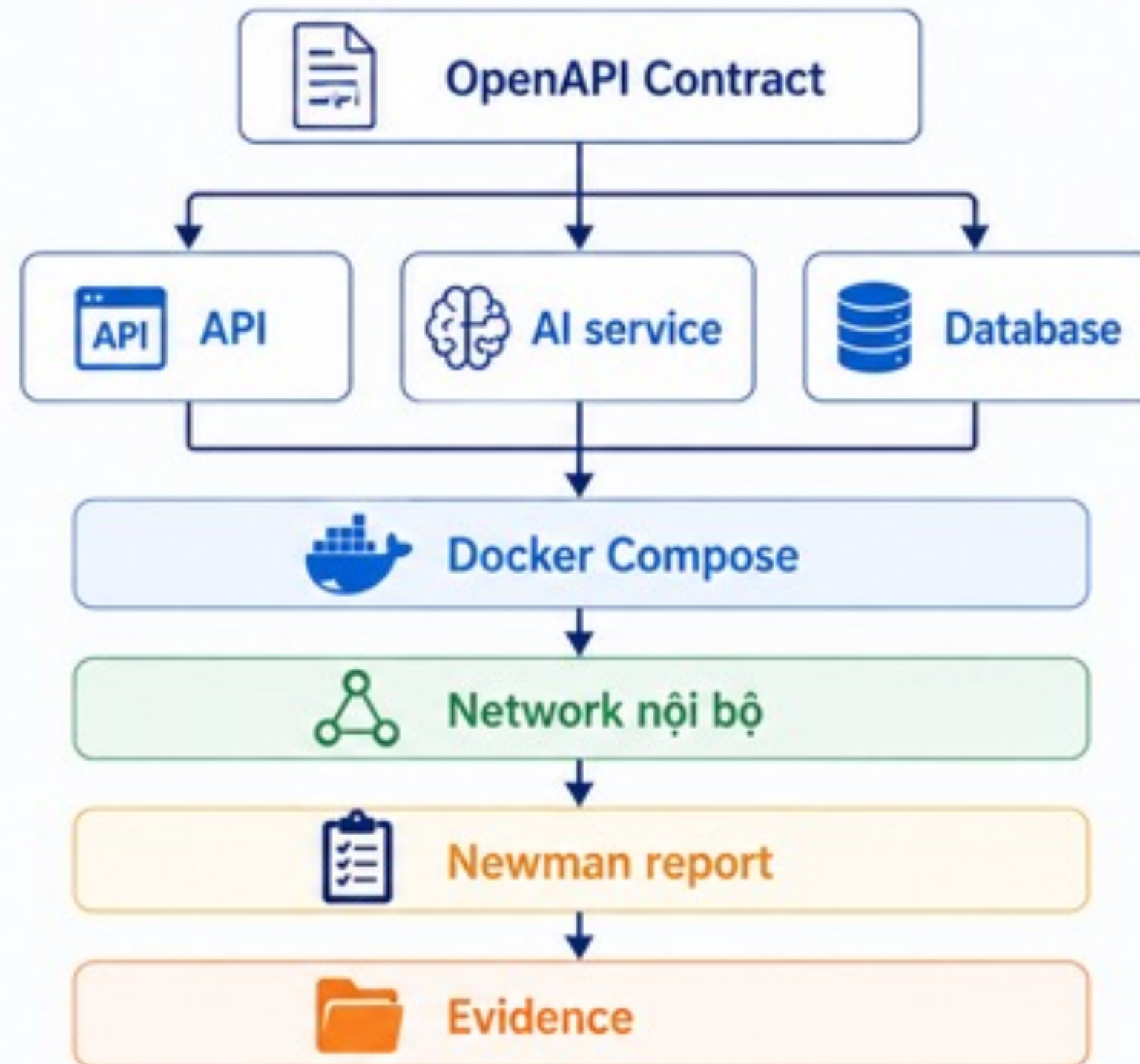
ĐIỀU PHỐI ĐA DỊCH VỤ TRONG SMART CAMPUS

Lab 04 – Một container



Mở rộng
tư duy
triển khai

Lab 05 – Nhiều service





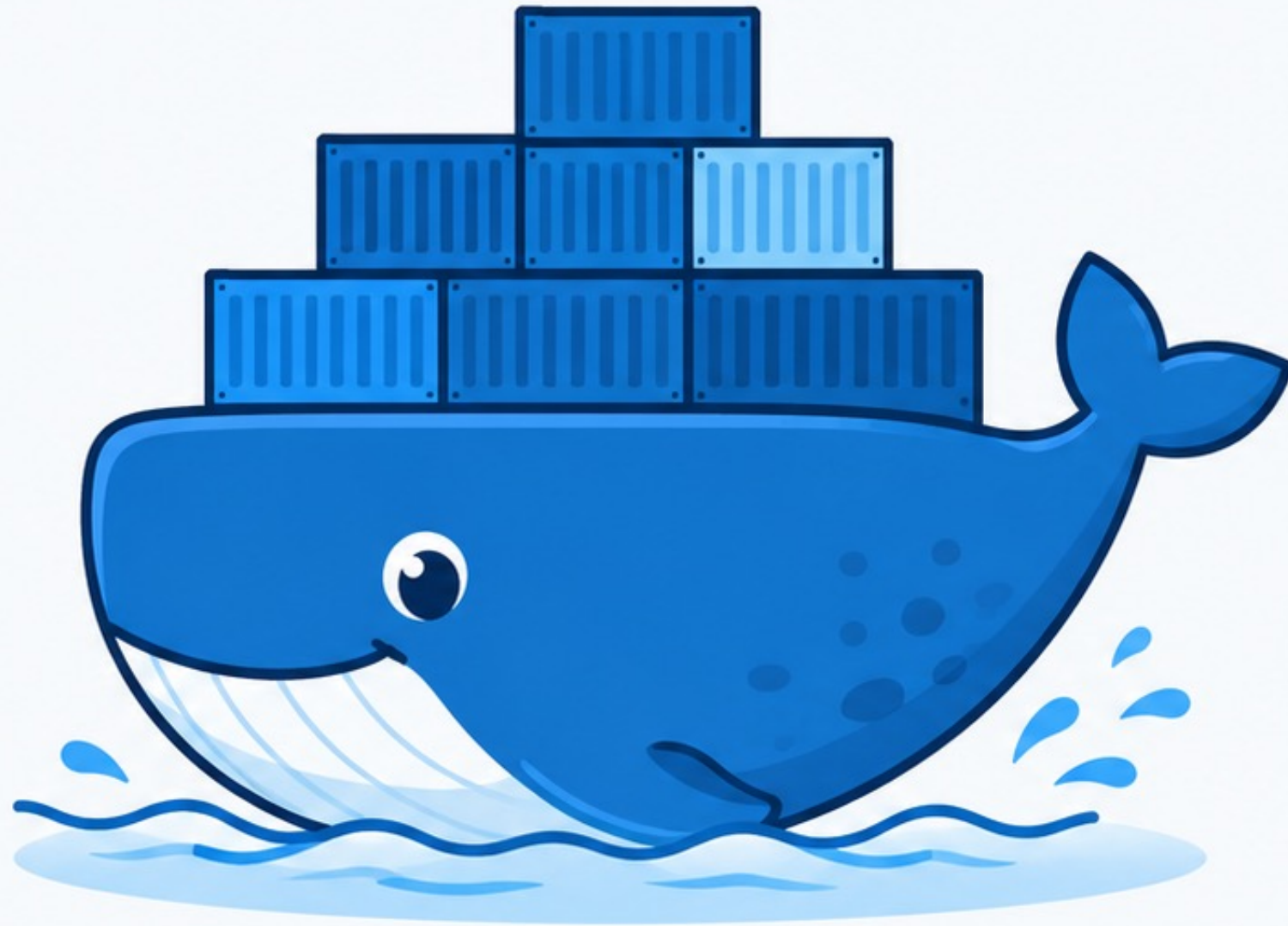
DOCKER COMPOSE

- ❖ **Docker Compose** là công cụ cho phép định nghĩa và chạy các ứng dụng Docker chứa nhiều container (multi-container).
- ❖ Thay vì phải gõ lệnh thủ công cho từng container, ta chỉ cần **cấu hình toàn bộ hệ thống bằng một file văn bản duy nhất** (thường là docker-compose.yml) và chạy bằng một câu lệnh

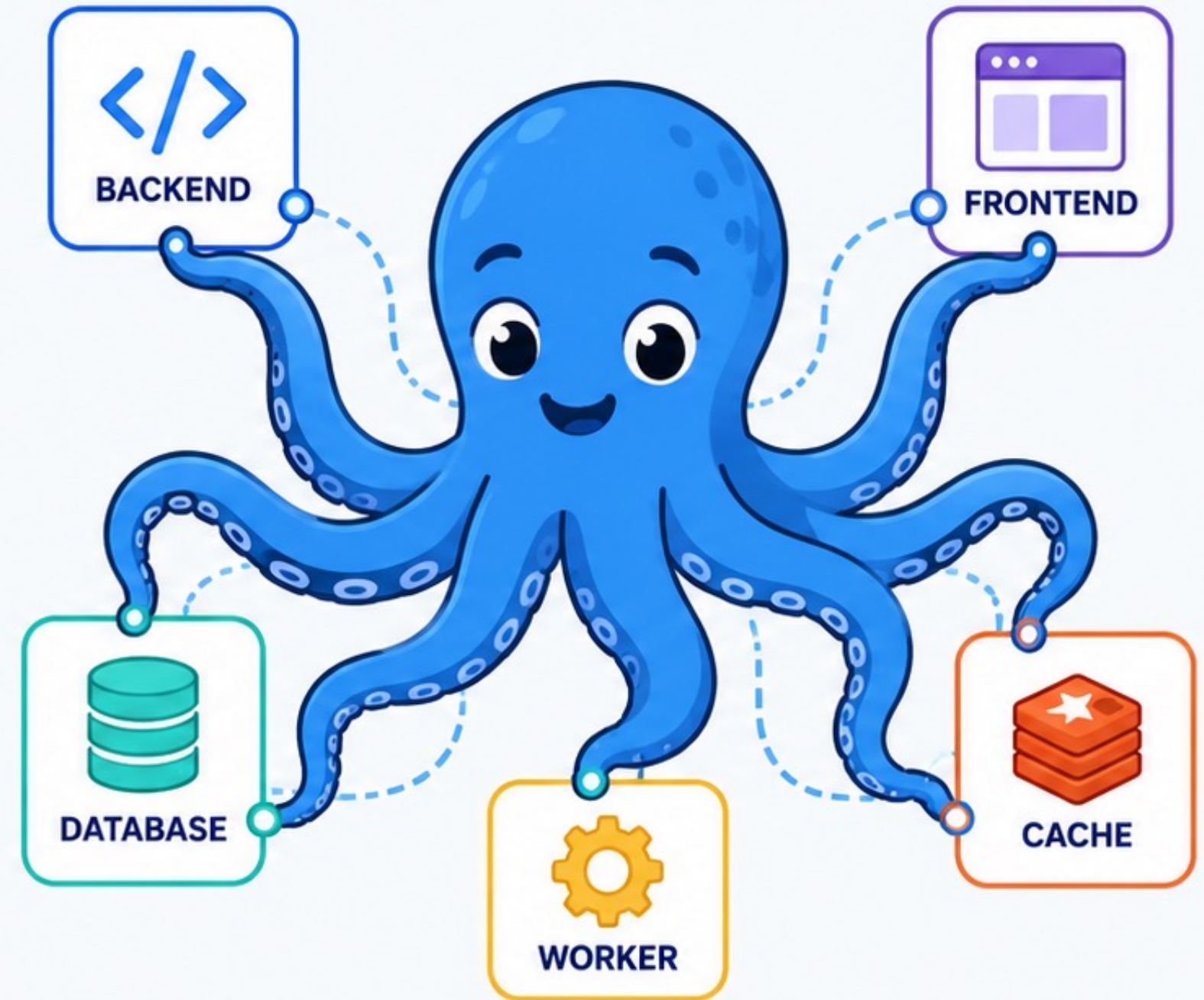


- ❖ **Docker Compose** is a tool for **defining and running multi-container** applications. It is the key to unlocking a streamlined and efficient development and deployment experience.
- ❖ Compose simplifies the control of your entire application stack, making it easy to manage services, networks, and volumes in a single YAML configuration file. Then, with a single command, you create and start all the services from your configuration file.
- ❖ Compose **works in all environments - production**, staging, development, testing, as well as CI workflows. It also has commands for managing the whole lifecycle of your application:
 - Start, stop, and rebuild services
 - View the status of running services
 - Stream the log output of running services
 - Run a one-off command on a service

DOCKERFILE VS DOCKER COMPOSE



Docker

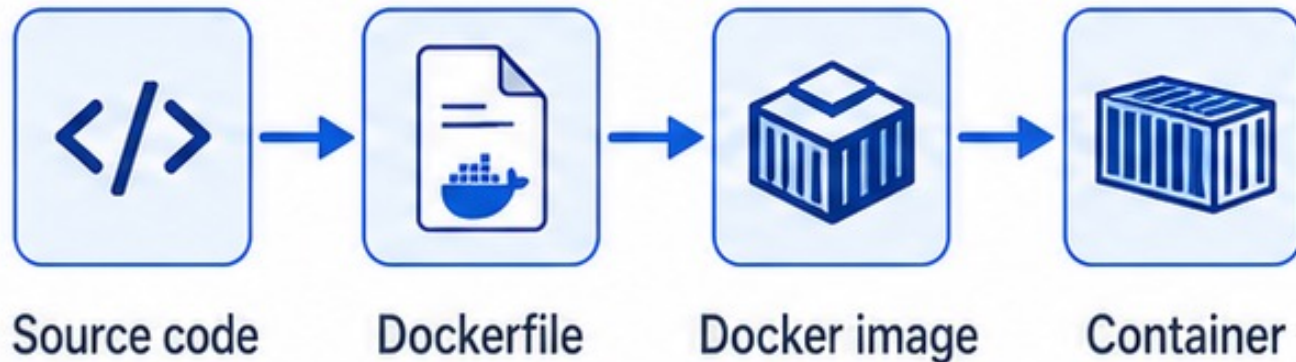


Docker Compose

DOCKERFILE VS DOCKER COMPOSE

Dockerfile

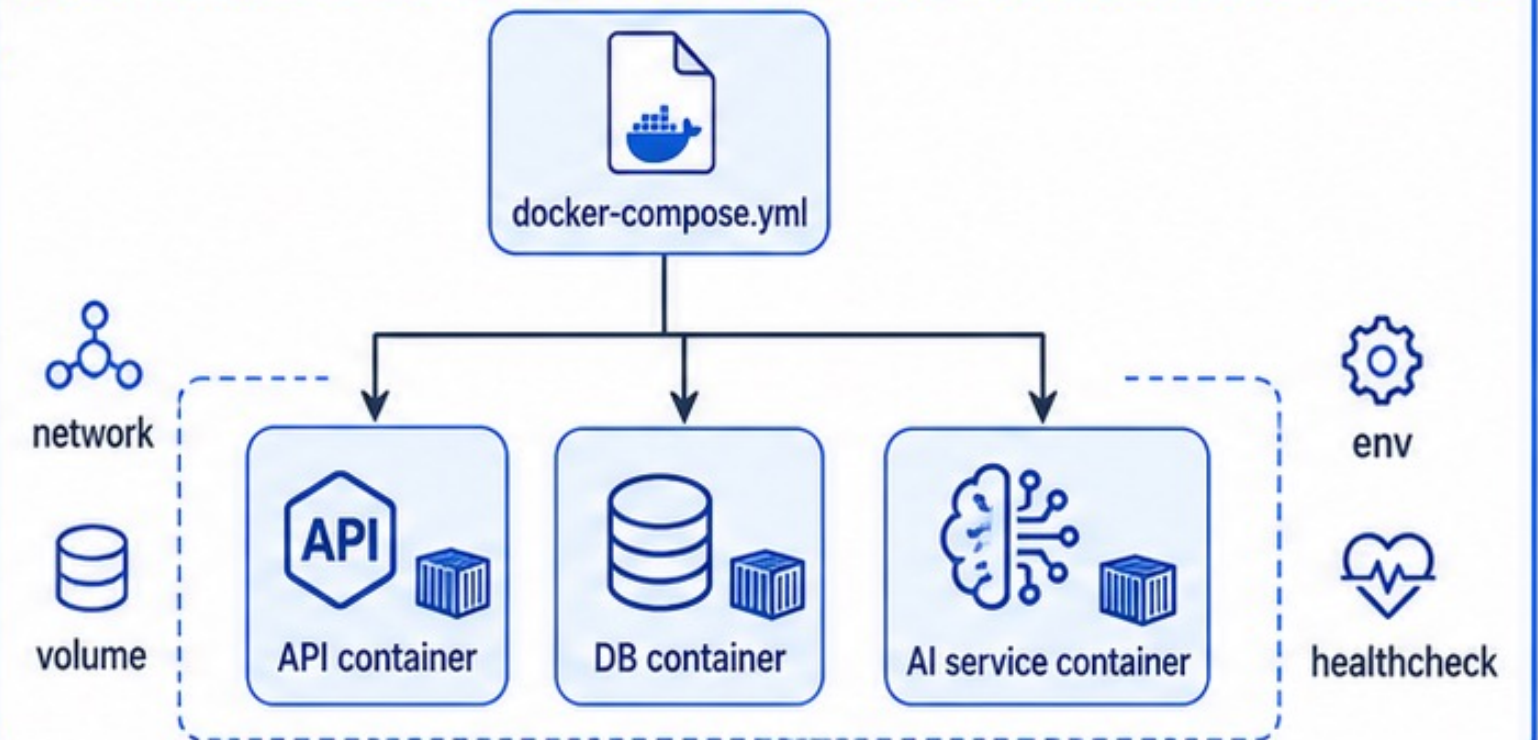
- Định nghĩa cách build một image
- Mô tả môi trường chạy của một service
- Tạo ra Docker image
- Dùng ở Lab 04



Từ một service
→
một stack
nhiều service

Docker Compose

- Định nghĩa cách chạy nhiều container cùng lúc
- Kết nối API, Database, AI service
- Quản lý network, volume, environment, healthcheck
- Dùng ở Lab 05



Dockerfile giúp **đóng gói service**. Docker Compose giúp **vận hành nhiều service** phối hợp với nhau.

- ❖ Các thành phần của docker compose:
 - **services:** Chỉ định image cần dùng (image hoặc build), cổng mạng (ports), biến môi trường (environment), và các tài nguyên liên quan như mạng, ổ cứng.
 - **container_name:** Cho đặt một tên cụ thể, cố định cho container (ví dụ: **container_name: my_database**). Việc này giúp dễ dàng thao tác gọi lệnh hoặc theo dõi log.
 - **networks:** định nghĩa các mạng ảo để các container có thể giao tiếp với nhau hoặc ra thế giới bên ngoài

- ❖ Các thành phần của docker compose:
 - **volumes**: Ánh xạ thư mục trên máy host (máy tính thật của bạn) hoặc tạo các vùng lưu trữ độc lập trên Docker để lưu dữ liệu vĩnh viễn (ví dụ: file database). Dữ liệu này sẽ tồn tại kể cả khi container bị xóa hoặc tạo lại.
 - **secrets**: Thay vì để mật khẩu dạng văn bản thô (plain-text) trong file cấu hình hay biến môi trường, **secrets** cho phép lưu các thông tin này vào các file cục bộ, sau đó mount an toàn vào trong container một cách mã hóa.
 - **profiles**: Gán một hoặc nhiều service vào một profile cụ thể. Khi chạy, nếu không chỉ định profile, những service này sẽ bị bỏ qua.

DOCKER COMPOSE

- ❖ Docker compose không chỉ chạy container, mà còn thể hiện quan hệ vận hành giữa các service



docker-compose.yml

```
version: '3.9'
services:
  api:
    image: my-api:latest
    ports:
      - "8080:80"
    environment:
      - DATABASE_URL=postgres://db:5432/app
      - APP_ENV=production
    depends_on:
      - db
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost/health"]
      interval: 30s
      timeout: 5s
      retries: 3
  networks:
    app-network:
      driver: bridge
  volumes:
    db-data:
      driver: local
```

1



services

Danh sách container cần chạy

2



ports

Map cổng container ra máy host

3



environment / env_file

Truyền cấu hình runtime

4



depends_on

Khai báo phụ thuộc giữa service

5



healthcheck

Kiểm tra service đã sẵn sàng chưa

6



networks / volumes

Kết nối nội bộ và lưu dữ liệu bền vững

SERVICE DISCOVERY & THỨ TỰ KHỞI ĐỘNG

⚡ Khi chạy từng container riêng lẻ... ⚡

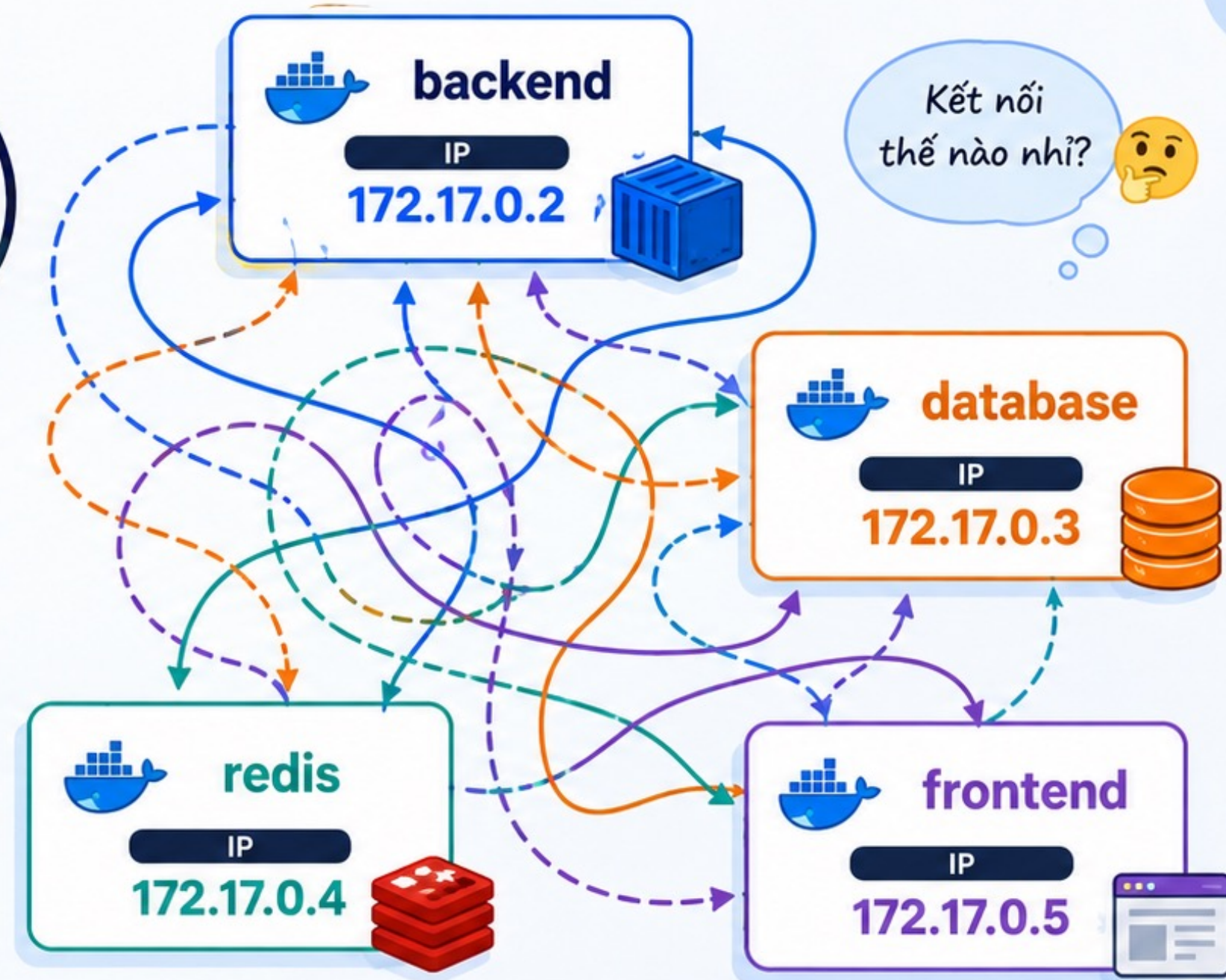


IP của container này là gì?

```
$ docker ps
```

CONTAINER ID	NAMES	STATUS	IP ADDRESS
b1a2c3d4e5f6	backend	Up 2 hours	172.17.0.2
c2b3d4e5f6a7	database	Up 2 hours	172.17.0.3
d3c4e5f6a7b8	redis	Up 2 hours	172.17.0.4
e4d5f6a7b8c9	frontend	Up 2 hours	172.17.0.5

Mỗi container một IP khác nhau!

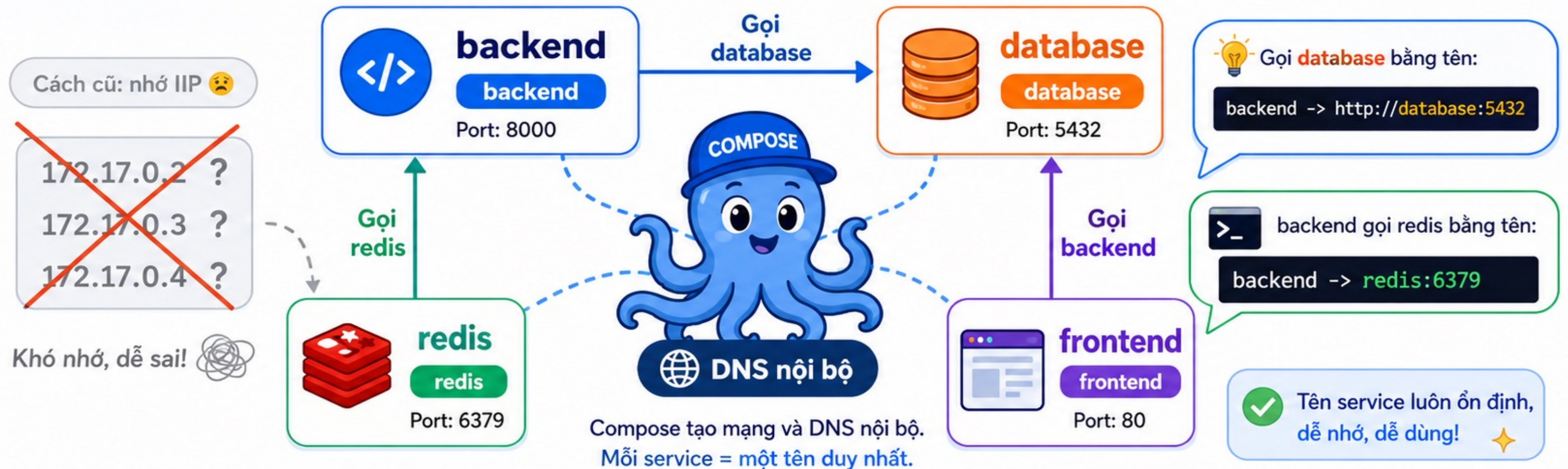


Hay phải **loay hoay** nhớ IP của từng container



Compose tạo DNS nội bộ

Mỗi service có một cái tên. Service khác chỉ cần gọi đúng tên đó.



Không cần nhớ IP — chỉ cần gọi **đúng tên service**

❖ Nếu service phụ thuộc khởi động trước khi DB sẵn sàng thì sao?



DEPENDS_ON

❖ Ví dụ:

```
db:  
  image: postgres:16  
  healthcheck:  
    test: ["CMD-SHELL", "pg_isready -U postgres"]
```

DB tự kiểm tra xem
mình đã sẵn sàng
nhận kết nối chưa

❖ Sau đó ở api:

```
api:  
  depends_on:  
    db: { condition: service_healthy }
```

API chỉ khởi động khi
DB đã báo healthy

READINESS CHECKLIST



1 Database readiness

- ✓ DB running
- ✓ pg_isready pass
- ✓ API kết nối được DB



2 AI service readiness

- ✓ /health trả 200
- ✓ model/mock đã sẵn sàng
- ✓ API gọi được ai-service



3 API readiness

- ✓ API running
- ✓ /health trả 200
- ✓ đọc đúng ENV



4 Network readiness

- ✓ team-internal hoạt động
- ✓ gọi bằng service name
- ✓ không dùng sai localhost



5 Security/config readiness

- ✓ có .env.example
- ✓ không commit secret
- ✓ không hard-code token



6 Evidence readiness

- ✓ có screenshot
- ✓ có log
- ✓ có Newman report

❖ Ví dụ:

```
services:
  api:
    networks: [team-net, class-net]

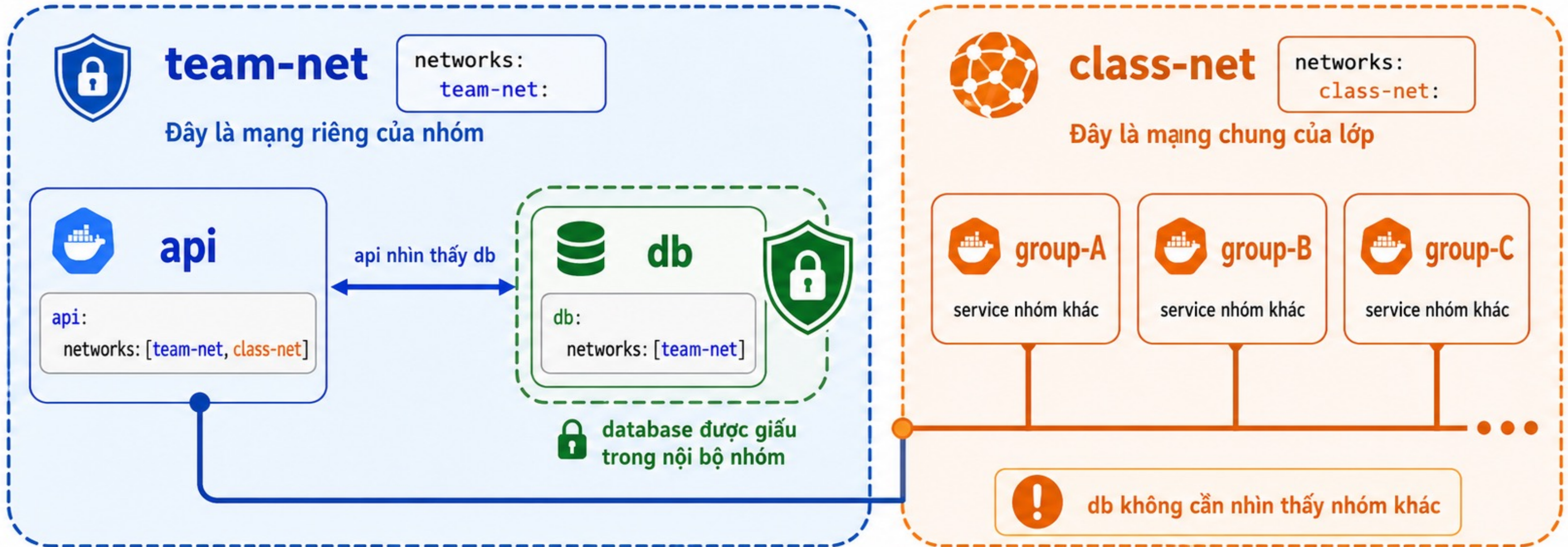
db:
  networks: [team-net]

networks:
  team-net:
  class-net:
    external: true
```

- Service nào đang đứng ở cả hai mạng? Vì sao?
- Service nào chỉ nằm trong team-net? Vì sao?



NETWORK TRONG DOCKER COMPOSE



- api nhìn thấy db
- db không cần nhìn thấy nhóm khác
- database được giấu trong nội bộ nhóm

- Kết nối trong team-net (mạng riêng của nhóm)
- Kết nối trong class-net (mạng chung của lớp)
- api là cầu nối giữa mạng riêng (**team-net**) và mạng chung (**class-net**)

QUY TẮC THIẾT KẾ

- ❖ Quy tắc 1: API mặt tiền được lên `class-net`

```
api:  
  networks: [team-net, class-net]
```

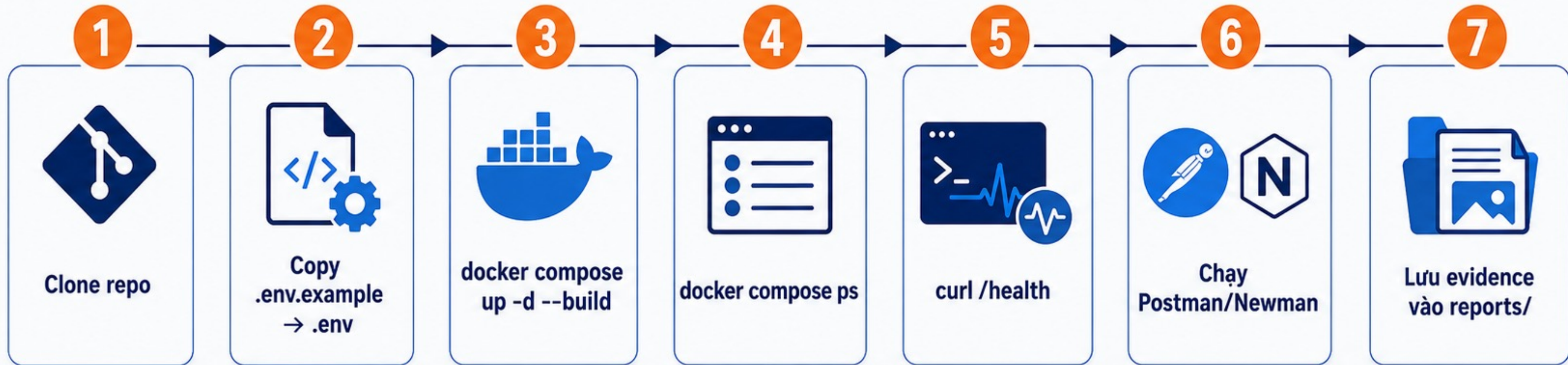
- ❖ Quy tắc 2: Database không lên `class-net`

```
networks: [team-net]
```

- ❖ Quy tắc 3: Worker/cache/service phụ cũng nên ở nội bộ
- ❖ Quy tắc 4: Nhóm khác chỉ nên phụ thuộc vào API contract

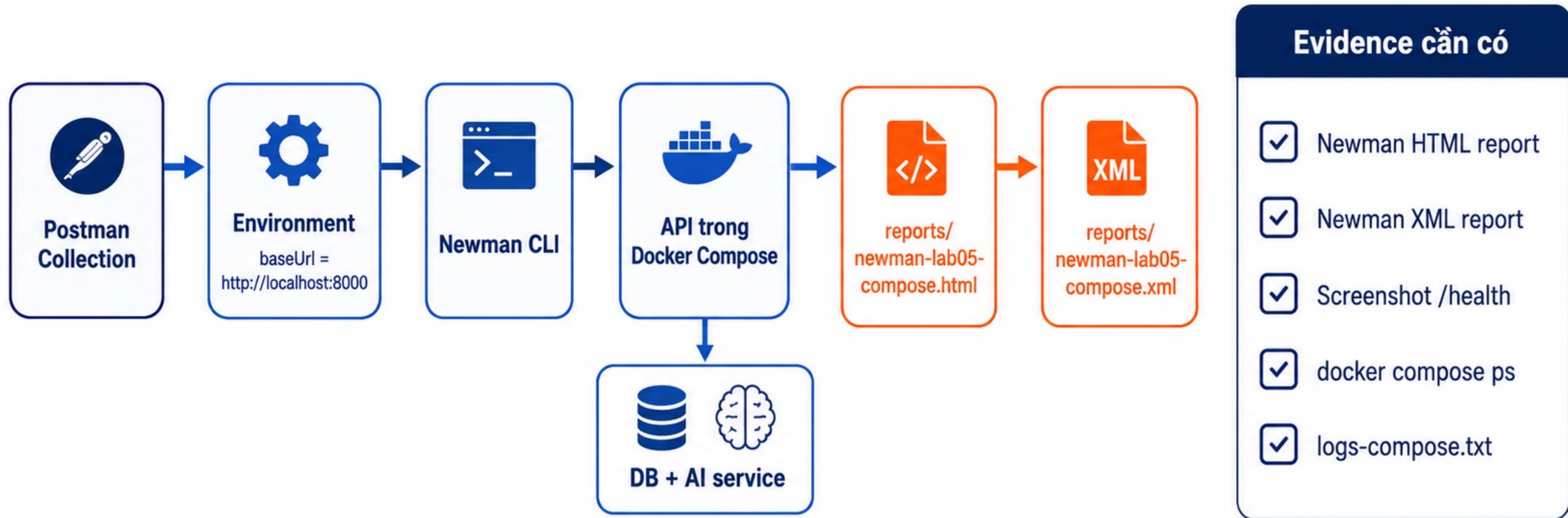


❖ Quy trình thực hành lab 5



Mục tiêu: người khác **clone repo** và chạy lại được **toàn bộ stack**.

KIỂM THỬ STACK COMPOSE BẰNG POSTMAN/NEWMAN



❖ Sản phẩm cần nộp của lab 5



Cấu hình triển khai

- ☒ docker-compose.yml
- ☒ Dockerfile
- ☒ .dockerignore
- ☒ .env.example
- ☒ Makefile



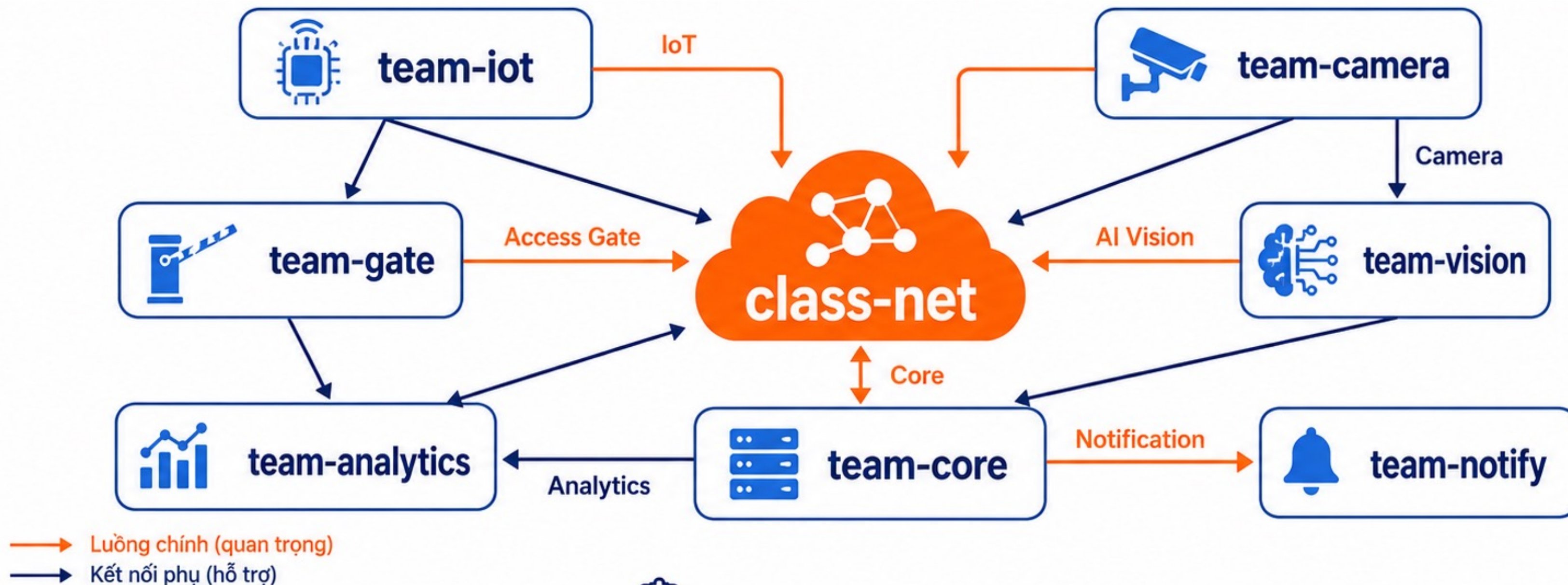
Kiểm thử và readiness

- ☒ Postman environment
- ☒ Newman report
- ☒ readiness-checklist.md
- ☒ screenshot /health
- ☒ logs-compose.txt



Tài liệu và bàn giao

- ☒ RUN_COMPOSE.md
- ☒ contracts/<team>.openapi.yaml
- ☒ tag image v0.1.0-<team>
- ☒ evidence trong reports/

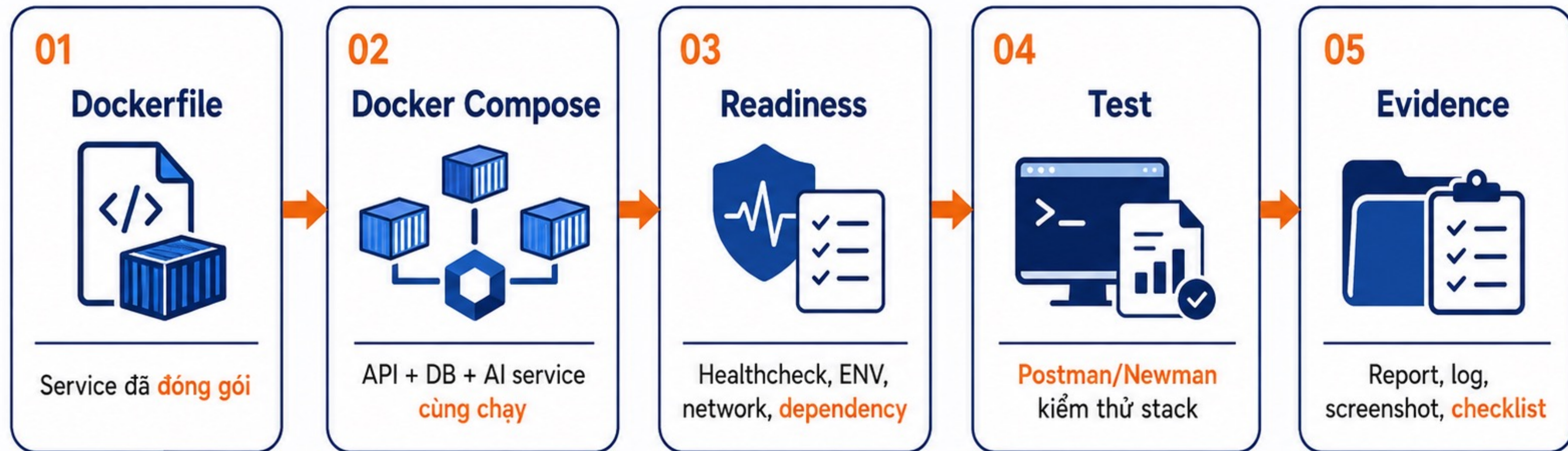


Checklist trước demo

- | | | | | | |
|-------------------|-------------------------|-----------------------------|---------------|--------------------|-----------------------------------|
| ✓ Service healthy | ✓ OpenAPI contract đúng | ✓ Gọi được qua service name | ✓ Có evidence | ✓ Có kịch bản demo | ✓ Có phương án xử lý lỗi upstream |
|-------------------|-------------------------|-----------------------------|---------------|--------------------|-----------------------------------|



Plug-a-thon kiểm tra khả năng **bắt tay thật** giữa các service.



**Sau Buổi 5, nhóm phải chứng minh hệ thống
nhiều service chạy được, kiểm thử được và sẵn sàng tích hợp.**



Thank You!

DAI NAM
UNIVERSITY